Ch#4 (Part 1)

Threading

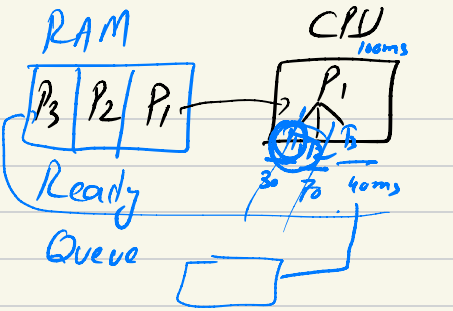
Agenda

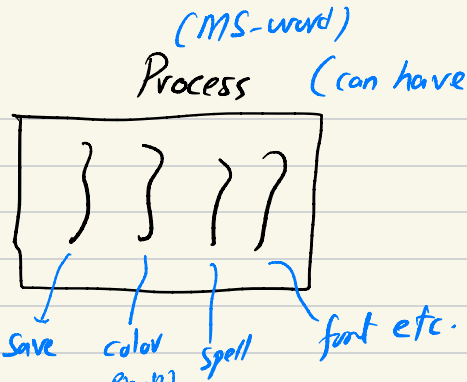
1. Overview (Definition + Types of threading Models)
2. Multicore Programming → Multicore ?? → Advantages

Overview

→ In order to implement / design Multiprocessing Application, developers use either
  → Parent-child Model
  → Threading

→ Since, threading is extremely fast & lightweight than Parent-child Model, therefore More Common & used as hybrid along-with other Models.





→ Each thread is responsible for a very specific task

→ Thread creation is also very lightweight as compare to Process creation (Most threads can be created without System Call)

→ Threads share code & data with each other (Provided by Process)

Definition of Thread

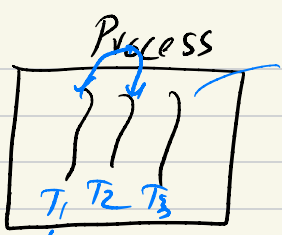
→ Basic Unit of CPU Execution

or

Lightweight Process that executes a (one) specific task.

→

Thread traits



CPU executes on thread at a time

Thread can have

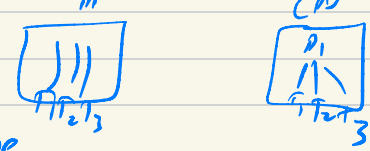
- ID
- Program Counter (Next Instruction)
- Set of Registers (Static)
- Stack (Temporary Variables Storage location)

Some threads have TCB ??

→ Threads Cannot Make System Calls, if thread needs some resources, it will request the Process.

→ Threads share resources & communicate with other threads using Process (No Kernel Module Needed / No IPC)

→ CPU executes only



one Process at a time but the context switching b/w thread is extremely fast (that apperently it seems simultaneous execution)

Benefits of threads

→ Responsiveness:- Hyper threaded Processes are more responsive / fast

→ Since, Context switching is really fast & if one of the threads is blocking, the Process schedules other threads

→ Resource Sharing → No IPC Needed, So, resources sharing b/w threads occur through internal Process Model

→ easy & fast

→ Economy → Lightweight on Memory & Compute

→ No Seperte System Call API Needed

→ Memory Allocation (Process Memory)

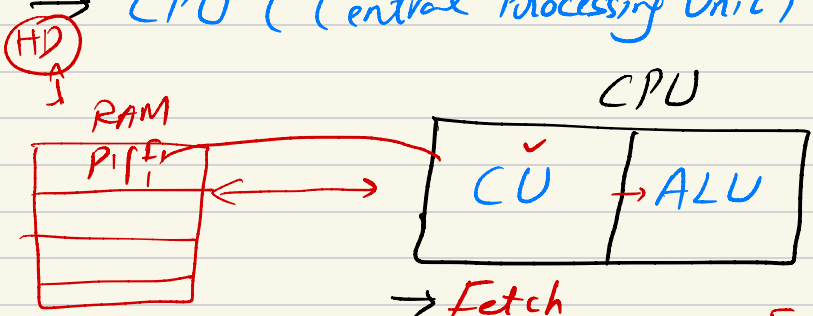
→ Scalibility → Compatible with Modern Hyperthreaded Modules/ Processor

Multicore Programming

CU → Control Unit

ALU → Arithmatic Logic Unit

→ CPU (Central Processing Unit)



→ Fetch Instructions → Execute Decoded instructions

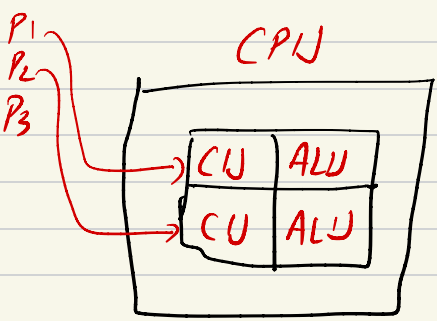
→ Decode instructions

Multi Core vs Single Core

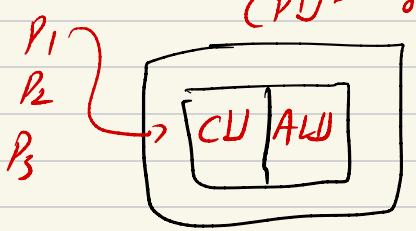
↓

CPU with Multiple CUs ALU

No. of CUs/ALU



CPU → with single/one CU/ALU



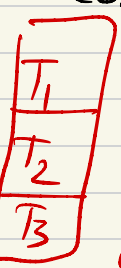
Advantages

→ Divide Activities

→ Balance b/w Cores (Data / Process Splitting)

→ Faster Execution, Debugging

Concurrency (Single Core) vs Parallelism (Multicore)



CPU

CU/ALU

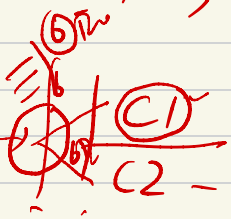
→ Act of Running Multiple Processes (Computations simultaneously)

→ Single Core Processor

→ Running Multiple tasks at the same time (Context Switching)

(Practically one task at a time) (1 GHz) → 1 Binst/sec

wmic → windows Manage Instr — P1



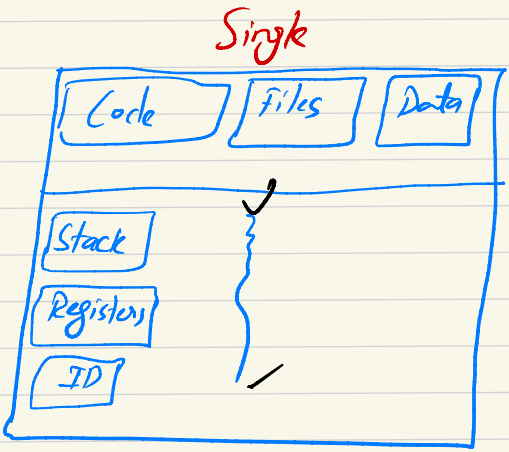
CPU Get NumberOfCores,

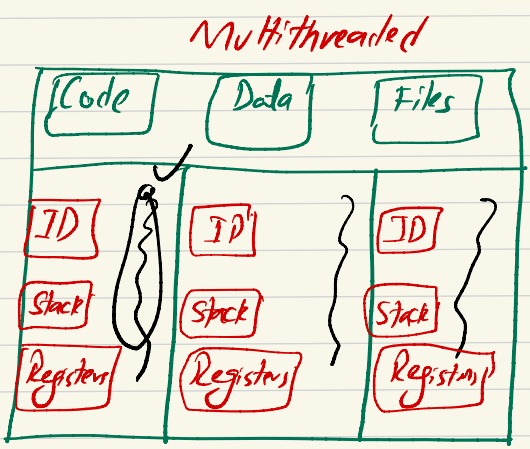
NumberOfLogicalProcessors

Types of threaded Processes

→ Single threaded Processes

→ Multithreaded Processes





→ Single thread in a Process

→ Each thread has its own ID, & Registers (Static) which they don't share with other threads.

→ However, they do share Codes, files of Process

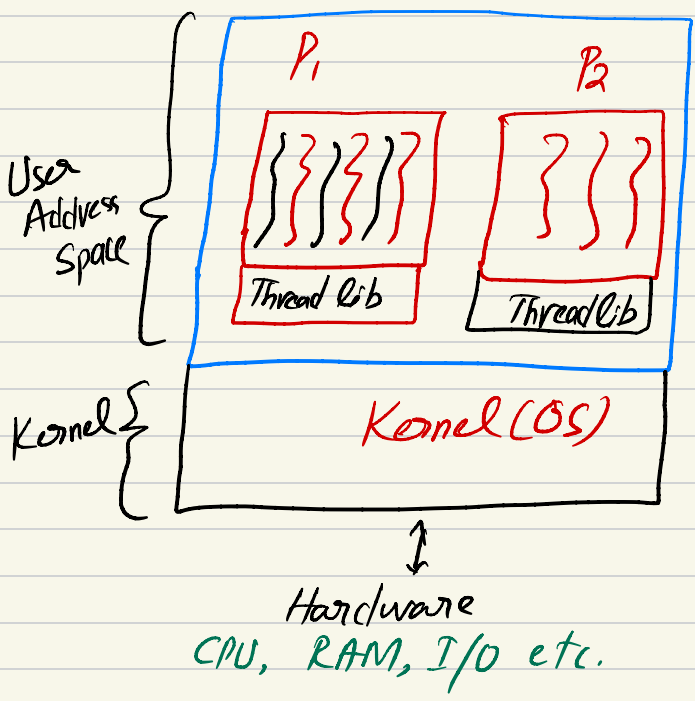
Threads Cannot Make System Call

→ But they can create child threads through Process

Types of Threads

- ULT (User Level Thread)
- KLT (Kernel Level Thread)

(a) User Level Threads



→ Applications / developers create threads using thread libraries (which reside within User Address Space)

↳ Unknown to Kernel

→ Thread Library is Responsible for:-

- → Create & Terminate threads
- → Schedule Threads
- → Communication etc.

Pros

- → Fast & effcient way to create threads
- → ULT don't Need usually Kernel level Privilege
- → Lightweight on Memory

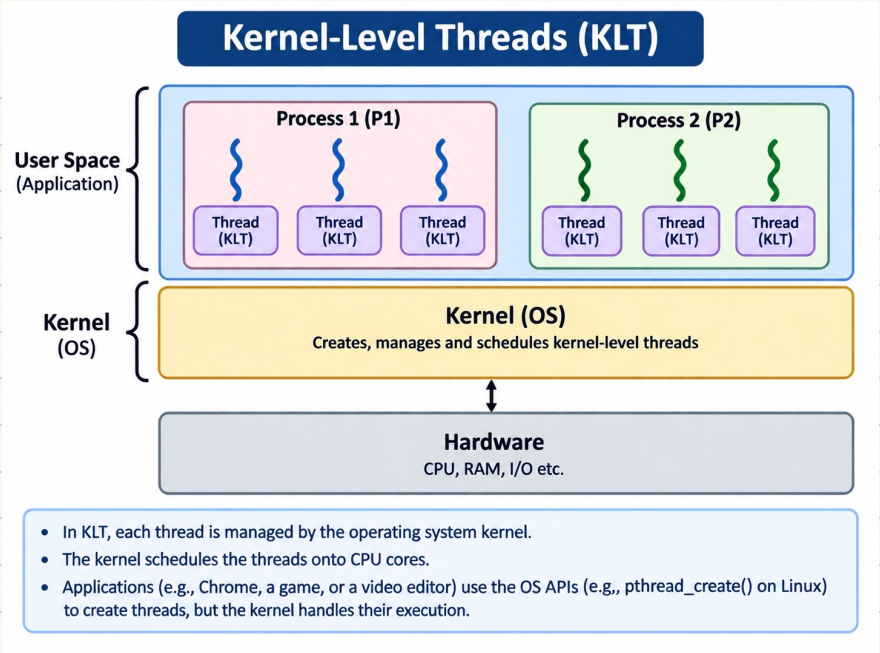
Cons

→ Lack of co-ordination b/w CPU & threads, if one of threads is blocking, then CPU moves the Process to blocking state.

→ Since, Kernel/CPU doesn't know about threading, So, Process as a while gets the time slice, regardless of the size/nb. of threads of Process.

KLTs

→ Kernel Level Threads



→ Kernel Threads are created & Managed by Kernel

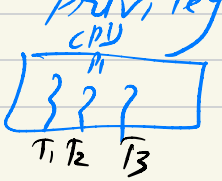
→ If OS supports threading at Kernel level → OS + HW Architecture are Hyperthread

→ Process requests/makes System Call each time to create KLT

→ Manage/Create/Delete etc. → KLT requires Kernel level Privileges.

KLT → Mode bit = 0

ULT → = 1



Pros

→ Unlike ULTs, In KLTs if any one of threads is blocking then because of Move Co-ordination b/w Kernel & Threads, it will let the Process to schedule other threads (Rather than blocking whole Process)

→ For optimization of Applications, the CPU can better scheduling time slice b/w Processes having KLTs

Cons

→ Slow (because for each KLT → System call)

→ Move Burden on RAM → (TCB) + Data Structure for each KLT

Ch#4 (Part 2)

Threading

Agenda

① Multithreaded Models (why??)
- Many to One (Uniprocessor)
- One to One (Multi Core)
- Many to Many (Single/Multi)

② Threading Issues
- System Calls?
- Signalling?! How to Address ??
- Thread Cancellation ??
- Thread Local Storage

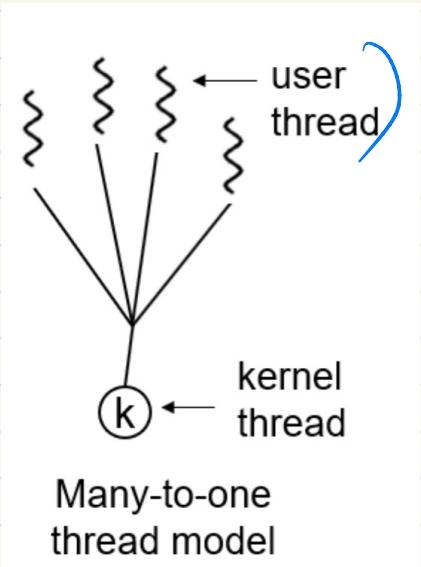
① Multithreaded Models ??

→ If an OS supports both ULTs & KLTs then to optimize/manage the Processes, it uses multithreaded Models (depends on your Processor → Single Core / Multi Core)

(i) Many to One

Many ULTs will be mapped onto single KLT

(ULTs will share this Single KLT



(Single Core)

→ Thread Management will be handled by Thread library → resides within User Address Space

→ No Parallelism (Single ULT ↓ takes Kernel Access at a time)

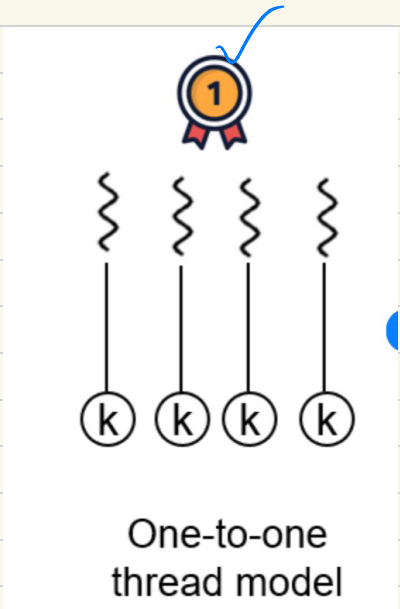
→ If one ULT is blocking then other have to wait

→ e.g. Solaris

One to One

Best Multithreading Model

e.g. Windows, MAC, Linux



→ For each ULT, there is KLT (Process will make System Call each time, it creates ULT.)

→ Each ULT will be mapped onto single KLT.

Pros:-

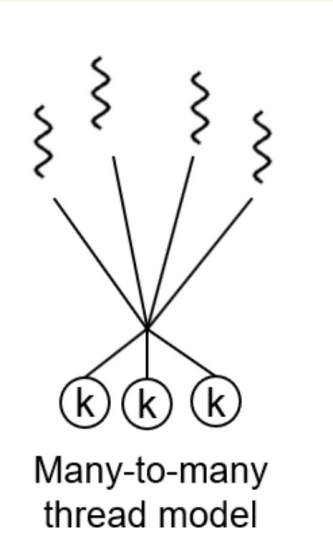
→ If one of threads is blocking, the other will continue executing.

Cons:-

↳ Slow but More Burden on RAM

Many to Many

ULT are mapped onto equal or less Number of KLTs



Many-to-many thread model

(User)

(Process defines the Threads Priorities)

→ Critical threads will have KLT while other thread may wait/share KLTs

→ Uni Processor vs Multi Processor

ULT > KLTs

ULT = KLT

Threading Issues

→ System Call?? ↔ fork() ↔ fork(1) / fork(2)
exec() → Whole Process (replaced)

→ Signal Handling?? ↔ Signal (Notification → CPU)
Interrupt (I/O)

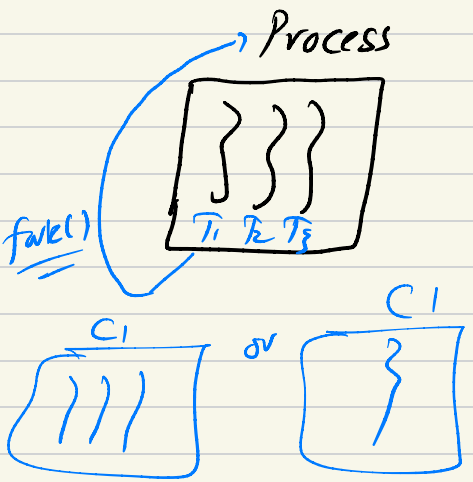
→ Thread Cancellation

→ Thread Local Storage

① System Calls

fork() :- Duplicates a Process (Create a Child Process)
→ New Process will have ID + PCB

Exec() :- Replaces the contents of one Process with another
P_1 = execute(P_2)



Question:-

→ Does New Process duplicate all threads?

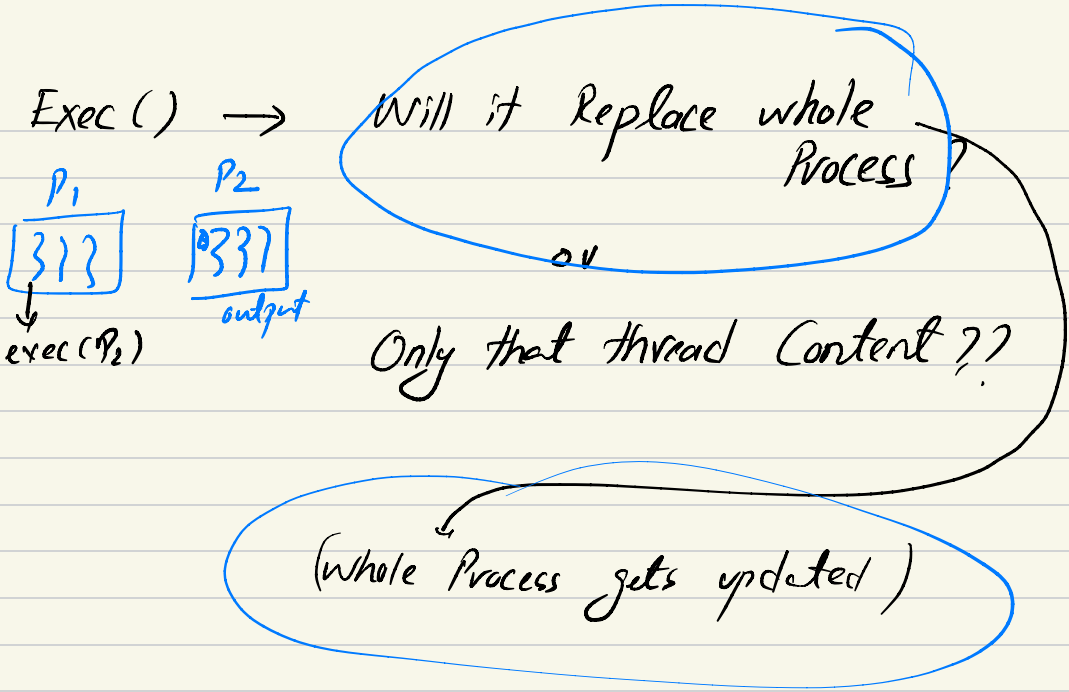
or

New Process will be Single threaded (thread invoked that system call)!

Ans:-

→ fork(1) → Creates a child Process with all the threads

→ fork(2) → Creates Single threaded Process



② Signal Handling

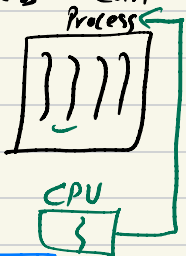
CNTL+C

Interrupt vs Signal

- → Events generated by external Components other than CPU
- → Upon receiving interrupt
- CPU Suspends on-going execution & gets interrupted task done first
- → Asynchronous Signal

- → Synchronous
- → Urgent Notification / User + Process
- → Event triggered by CPU/Program running on it

→ Signal is used to notify the Process/ User that a particular event has occurred, so Process can further Proceed / Terminate.



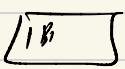
Signal Handlers

Default Signal Handler

Rulebook for CPU (Safety)

→ CPU defines how to respond to a certain event caused by Process / Thread

16 bit



User Defined

→ Programmed by the developer of Process/ Program

→ Process (after receiving signal) decides how to handle it

Problems: -

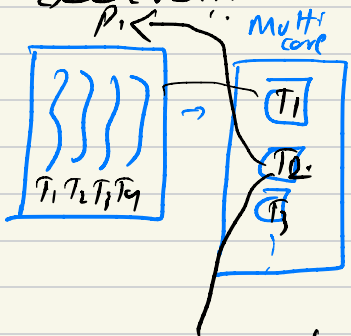
→ Where

Question:- Where this Signal will delievered??

(i) Will it be delievered to Process

or

(ii) Specific Thread



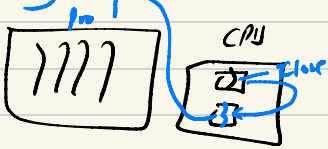
Blocked / Problem (Memory error)

Solution:-

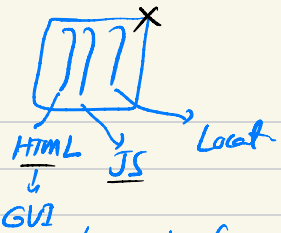
→ Synchronous (KLTs)/Multithreaded Model

↳ delievered to Specific thread that Specific thread can be terminated / blocked

Asynchronous → Process will be terminated by OS

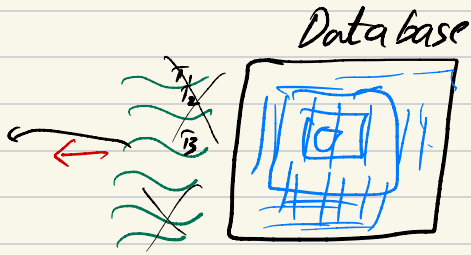


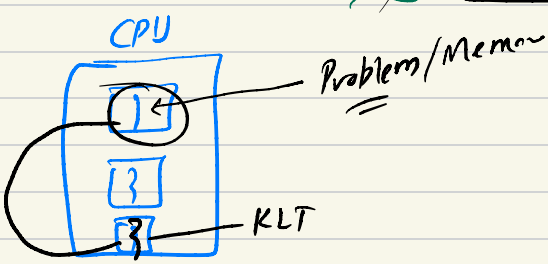
③ Thread Cancellation



→ Task of Terminating a thread before it has finished executing its statements.

Why do we Need to Cancel a Thread??





Target Thread

Victim Thread
-, A thread that needs to be cancelled

Cancellation Types

Asynchronous Cancellation ✓

→ OS appoints/create KLT who cancels the Target thread immediately

Problem :—

→ Deffered Cancellation

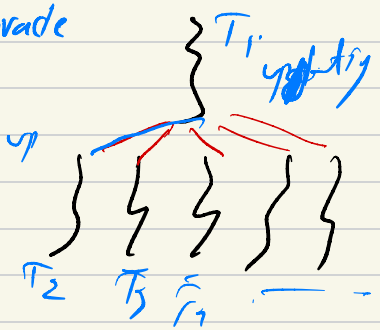
Deffered Cancellation

→ Target thread will periodically checks & self terminate whenever

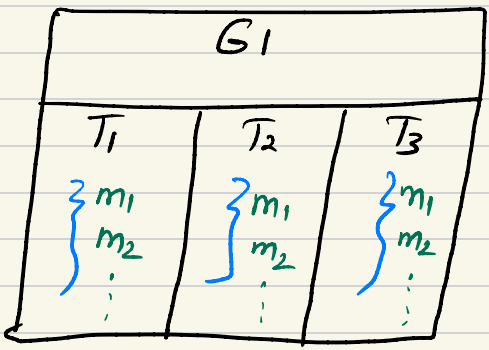
① it has finished the tasks / safe to do so

→ Return all resources to OS

sudo apt upgrade



Thread Local Storage:-



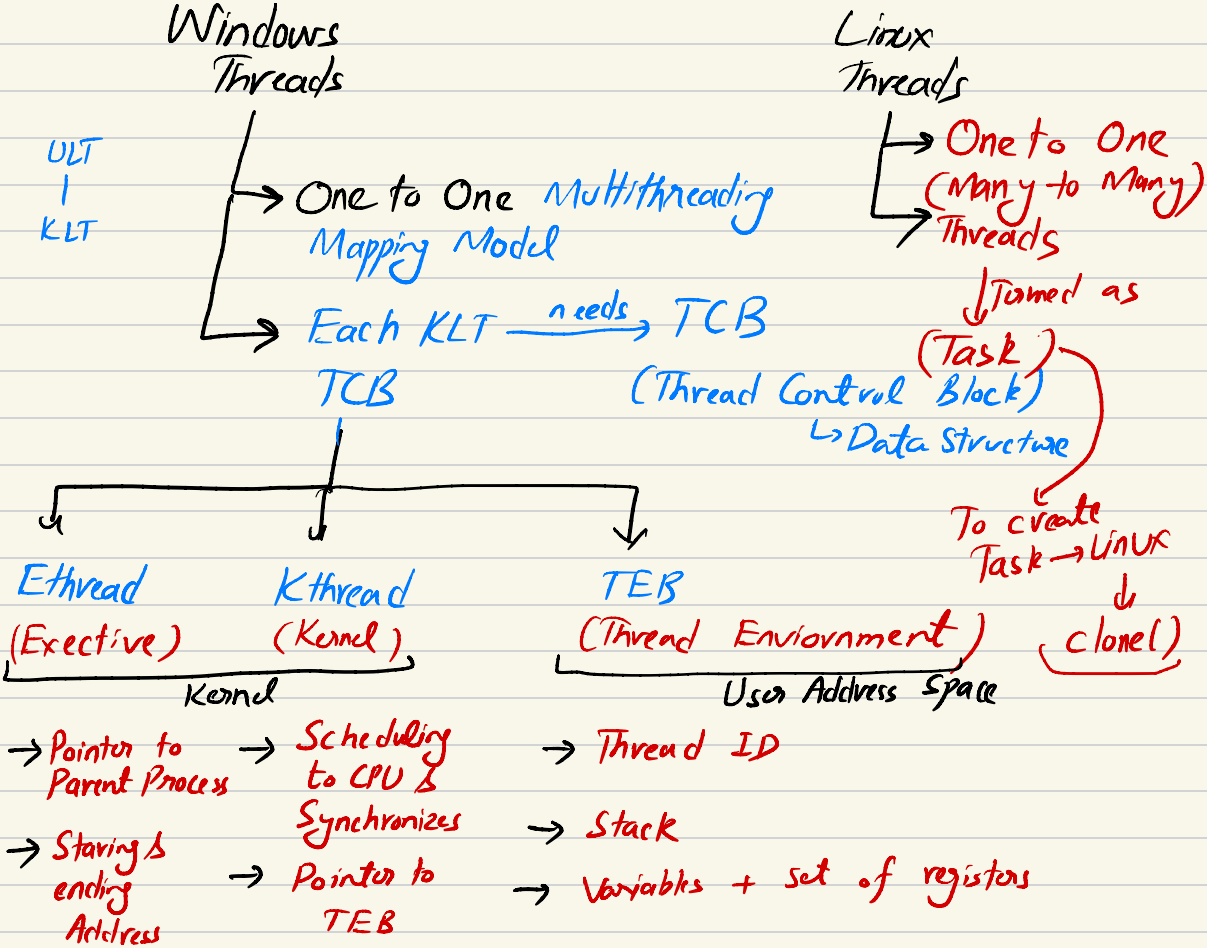
$m_1, m_2 \ldots$ ⟶ Local Thread variable

G1.. ⟶ Global Process variable which will be shared among threads

Process:- Threads loads the instruction onto local variables & request for Global Variable (of the Process), through Global variable, Process make System Call & Pass instructions to CPU

→ If two or more threads access G1 simultaneously, it may create Deadlock

→ To avoid deadlocks, hyper threaded Models allocate dedicate Global variable


Windows Threads
ULT | KLT
One to One Multithreading Mapping Model
Each KLT needs TCB
TCB
Linux Threads
One to One
(Many to Many)
Threads
Termed as
(Task)
(Thread Control Block)
↳ Data Structure
To create Task → Linux
clone()
Ethread
(Exective)
Kthread
(Kernel)
Kernel
TEB
(Thread Enviornment)
User Address Space
→ Pointer to Parent Process
→ Starting & ending Address
→ Scheduling to CPU & Synchronizes
→ Pointer to TEB
→ Thread ID
→ Stack
→ Variables + set of registers